A memória na Máquina Virtual Java (JVM) é dividida em diferentes áreas de runtime para gerenciar a execução do programa e o armazenamento de dados: [1]

- **Heap (Memória Heap)**: Armazena todas as **instâncias de objetos e arrays** criados em tempo de execução. É uma única área compartilhada por todas as threads da aplicação e é dividida em Geração Jovem (*Young Generation*) e Geração Antiga (*Old Generation*). O gerenciamento e a limpeza de objetos sem referência nesta área são feitos automaticamente pelo **Garbage Collector**. [1, 2]
- **Stack (Pilha de Memória)**: Conhecida como *JVM Stack*, é uma área **privada de cada thread**. Ela armazena as variáveis locais (incluindo valores de tipos primitivos) e as referências para os objetos que estão no Heap, além de registrar a pilha de execução de chamadas de métodos. Cada vez que um método é chamado, um novo bloco (*frame*) é empilhado e removido assim que o método termina. [1, 2]
- **Metaspace**: Armazena os **metadados das classes**, como a estrutura de bytecode, nomes de métodos, anotações e constantes. Localiza-se na memória nativa do sistema operacional (fora do Heap) e substituiu a antiga área *PermGen* a partir do Java 8. [1, 2]
- **PC Register (Program Counter Register)**: Cada thread possui o seu próprio registrador de contador de programa, que armazena o **endereço da instrução de bytecode** que está sendo executada no momento pela thread. [1, 2]
- **Native Method Stack**: Semelhante à *Stack* tradicional, mas dedicada exclusivamente ao suporte de **métodos nativos** (escritos em outras linguagens, como C ou C++, e chamados via JNI - *Java Native Interface*). [1, 2]
- **Code Cache**: Uma área de memória usada pelo compilador JIT (*Just-In-Time*) da JVM para armazenar o **código de máquina nativo** gerado a partir do bytecode, acelerando a execução de trech